ZAPIER GUIDE

Meeting Brief — Zapier Implementation Guide

Workflow ID: 02-meeting-brief • Backstory public rollout asset

This file is a plain-English implementation guide. It is not an importable JSON artifact.

Zapier does not accept a reusable "upload this workflow JSON" format for the kind of cross-workspace assets this catalog is describing. To replicate this workflow in Zapier, build it as one or more Zaps plus, when needed, a custom Zapier app. If you want a reusable public Zap Template, every integration in the template must be a public integration, and Zapier rejects templates that include Code, custom Webhook, or custom Formatter steps. If you are creating workflows through Zapier workflow-creation APIs, use Public Apps because private apps are not supported there.

Workflow Summary

- Workflow ID: `02-meeting-brief`
- Category: daily-intelligence
- Source trigger in this catalog: Sub-workflow (called by Meeting Prep Cron, every 15 min) |
- Recommended Zapier trigger surface: Sub-Zap orchestration via Zapier Tables, Interfaces, or a parent app trigger
- Delivery targets: Slack channel or direct message, Microsoft Teams channel or chat, Email delivery, Calendar event or task, Webhook, queue, database, or downstream workflow sink

What To Build In Zapier

- Published Backstory custom Zapier app with explicit search and create actions
- Slack app actions for channel posts and direct messages
- Microsoft Teams app actions or companion Microsoft 365 app steps
- Gmail, Microsoft Outlook, or Email by Zapier actions
- Google Calendar or Microsoft Outlook Calendar actions
- Meeting-system native app or published custom Zapier app for transcript retrieval
- Zapier Tables or Storage for state, routing maps, and dedupe keys
- Approved webhook sink, database action, queue adapter, or downstream custom app action for publication

Recommended Implementation Shape

1. Publish the Backstory integration as a custom Zapier app before you try to templatize the workflow. Use the Zapier Platform UI or CLI if you need custom triggers or actions.
2. Create the primary Zap with the trigger and the minimum deterministic steps needed to gather source data and reach structured synthesis.
3. Split delivery, approvals, or fan-out into additional Zaps when the workflow naturally decomposes or when public-template restrictions would block a single-Zap design.
4. Use Zapier Tables or Storage for routing maps, dedupe keys, approval state, or cross-Zap handoff instead of hiding state in code.
5. If you plan to publish a public Zap Template, test the template path separately from any internal/private workspace build.

Custom App Actions To Provide

- `backstory_list_source_records`
- `backstory_get_record_context`
- `backstory_publish_run_summary`
- `meeting_source_fetch_artifact`
- `load_routing_map`

Zaps To Create

- `02_meeting_brief_primary_zap` — Trigger: Sub-Zap orchestration via Zapier Tables, Interfaces, or a parent app trigger. Native steps: Backstory custom Zapier app action: enrich_with_account_context -> AI step backed by a native LLM app returning strict JSON for AI Brief Generation -> Native delivery step: Gmail, Outlook, or Email by Zapier action for Deliver via Messaging. Notes: Keep the primary Zap focused on normalization, source access, and structured synthesis. Route secondary fan-out through additional native Zaps when template restrictions or retry requirements demand decomposition.
- `02_meeting_brief_delivery_or_followup_zap` — Trigger: Zapier Tables state change, Interfaces action, or an upstream custom app event. Native steps: Read delivery_payload and target metadata from structured fields -> Slack app action: Send Channel Message or Send Direct Message -> Microsoft Teams app action or Microsoft 365 companion action -> Gmail, Outlook, or Email by Zapier action -> Google Calendar or Microsoft Outlook Calendar action -> Approved webhook sink, database action, queue adapter, or downstream custom app action. Notes: Use this secondary Zap when you need reusable delivery fan-out, fallback paths, or approval boundaries without falling back to Webhooks or Code steps.

Contracts To Preserve

- `run_context`: workflow-level execution context such as trigger, mode, lookback, delivery mode, and dry-run state.
- `source_record`: normalized business record from the source adapter or source-system action layer.
- `enrichment_context`: MCP or native app enrichment results used only during synthesis.
- `delivery_payload`: deterministic delivery fields for the final native connector or app action.

Structured AI Requirements

- `ai_brief_generation`: return JSON only with keys `title`, `summary`, `confidence`, `recommended_actions`, `source_refs`. Intent: AI Agent analyzes the account context and composes a structured briefing with key talking points, recent interactions, and risk/opportunity signals.

Validation Checklist

- Do not treat this guide as an importable JSON workflow; build the workflow as one or more Zaps and, if needed, a custom Zapier app.
- If you want a reusable public Zap Template, only public integrations can be used in the template.
- Do not include Code, custom Webhook, or custom Formatter steps in a public Zap Template.
- If you are creating workflows through Zapier workflow-creation APIs, use Public Apps only and avoid Paths because they are not currently supported there.
- Validate structured AI output before any Slack, Teams, email, calendar, Jira, or sink action runs.

Official References

- Zap templates restrictions
- Known limitations for workflow creation
- Build with CLI
- Zapier action design
- Add a create action
- AI actions
- Custom actions and API requests actions